MATERIAL COMPLEMENTAR / GUIA TÉCNICO EP

Widgets e Interatividade

Componentes, eventos, sincronização de estado, visualizações reativas e depuração de interfaces no Jupyter.

Propósito — servir como material de estudo aprofundado e referência rápida, com exemplos originais, atividades e integração com os notebooks.

Como utilizar — leia os conceitos, reproduza os exemplos no ambiente indicado e compare as atividades com o notebook correspondente. A consulta rápida aparece nas seções finais.

ROTEIRO DO DOCUMENTO

1. Da célula estática à interface interativa
2. Famílias de controles e propriedades
3. Interact: prototipagem simples
4. Observe: escutar mudanças de estado
5. Botões, Output e processamento controlado
6. Estado e sincronização entre controles
7. Layout: clareza e acessibilidade
8. Filtragem de dados e renderização reativa
9. Uploads, entrada do usuário e segurança
10. Erros comuns e diagnóstico
11. Testes: lógica separada da interface
12. Consulta rápida: APIs mais frequentes
13. Oficina orientada: painel de exploração
14. Aprofundamento: desenho de uma aplicação reativa
15. Aprofundamento: desempenho, acessibilidade e portabilidade
16. Exercícios e leituras

1. Da célula estática à interface interativa

Um notebook executa células e mostra resultados. Com `ipywidgets`, o usuário também altera parâmetros por controles visuais, como seletores, sliders, botões e caixas de texto. O widget possui **estado Python** e uma representação gráfica no navegador; a comunicação entre esses lados é o que permite respostas reativas.

Nem todo valor deve ser interativo. Um widget é apropriado quando ajuda a explorar sensibilidade de um modelo, filtrar resultados ou ensinar relações entre parâmetros. Se muitos controles forem necessários, talvez seja melhor criar uma aplicação dedicada.

Inspeção inicial

```
import ipywidgets as widgets
from IPython.display import display

controle = widgets.IntSlider(value=5, min=0, max=10,
                             step=1, description='Limite')
display(controle)
print('Valor no Python:', controle.value)
```

Se aparecer apenas uma representação textual, o kernel pode estar funcionando enquanto a extensão de interface não está carregada. Não confunda falha de renderização com erro do Python.

2. Famílias de controles e propriedades

Categoria	Exemplos	Uso didático
Numéricos	`IntSlider`, `FloatSlider`, `IntText`	Sensibilidade a parâmetros
Seleção	`Dropdown`, `RadioButtons`, `SelectMultiple`	Filtrar categorias
Booleanos	`Checkbox`, `ToggleButton`	Ativar comportamentos
Ação	`Button`	Disparar processamento explícito
Texto	`Text`, `Textarea`	Receber filtros e comentários
Apresentação	`HTML`, `Output`, `Label`	Informações e resultados
Layout	`VBox`, `HBox`, `GridBox`, `Tab`	Organizar componentes

Atributos como `value`, `description`, `disabled`, `min`, `max` e `options` podem ser inspecionados e, em muitos casos, alterados dinamicamente. Restrinja valores com limites coerentes para evitar estados inválidos.

3. Interact: prototipagem simples

`interact` associa parâmetros de função a controles e executa a função quando valores mudam. É adequado para demonstrações curtas, mas não para tarefas caras disparadas repetidamente.

```
from ipywidgets import interact, IntSlider

def potencia(base, expoente):
    print(f'{base}^{expoente} = {base ** expoente}')

interact(potencia,
         base=IntSlider(value=2, min=1, max=10),
         expoente=IntSlider(value=3, min=0, max=8))
```

Para cálculos demorados, use botão ou atualização deliberada. Não execute processos intensivos a cada movimento do slider.

4. Observe: escutar mudanças de estado

`observe` registra uma função de retorno que recebe um dicionário de mudança. O campo `new` contém o novo valor e `old`, o anterior. Filtrar por `names='value'` evita reagir a propriedades irrelevantes.

```
import ipywidgets as widgets
from IPython.display import display

nivel = widgets.IntSlider(min=0, max=10, value=4)
resultado = widgets.HTML()

def ao_mudar(change):
    resultado.value = f'<b>Novo valor:</b> {change["new"]}'

nivel.observe(ao_mudar, names='value')
display(widgets.VBox([nivel, resultado]))
```

Cuidado: executar a célula de registro várias vezes pode acrescentar observadores duplicados. Em notebooks iterativos, mantenha a referência ao callback e use `unobserve` quando necessário.

5. Botões, Output e processamento controlado

Um botão produz eventos de clique; `Output` captura mensagens e representações geradas dentro de um bloco. Essa abordagem é útil quando uma operação depende de validação prévia.

```
import ipywidgets as widgets
from IPython.display import display

botao = widgets.Button(description='Calcular')
saida = widgets.Output()

def calcular(_):
    with saida:
        saida.clear_output(wait=True)
        print('Soma:', sum(range(1, 11)))

botao.on_click(calcular)
display(widgets.VBox([botao, saida]))
```

Uma interface robusta separa a lógica do cálculo do código que atualiza widgets. Isso facilita testes sem a necessidade de abrir o JupyterLab.

6. Estado e sincronização entre controles

`widgets.link` conecta duas propriedades em Python; `jslink` realiza uma ligação no lado da interface, quando suportada. A escolha depende da necessidade de comunicação com o kernel e das restrições de exibição.

```
import ipywidgets as widgets
from IPython.display import display

a = widgets.IntSlider(min=0, max=100, description='A')
b = widgets.IntText(description='B')
ligacao = widgets.link((a, 'value'), (b, 'value'))
display(widgets.VBox([a, b]))
# Ao terminar: ligacao.unlink()
```

Evite criar dependências cíclicas com callbacks manuais. Quando os dados mudam, defina uma única fonte de verdade para os valores relevantes.

7. Layout: clareza e acessibilidade

Use VBox para empilhamento vertical e HBox para disposição horizontal. Para interfaces maiores, separe filtros, resultados e feedback em áreas distintas. Rótulos devem explicar a ação e unidades: “Taxa (%)” é melhor que “Parâmetro”.

```
import ipywidgets as widgets
from IPython.display import display

regiao = widgets.Dropdown(options=['Todas','Norte','Sul'],
                          description='Região')
limiar = widgets.FloatSlider(value=0.5, min=0, max=1,
                             step=0.05, description='Limiar')
painel = widgets.VBox([
    widgets.HTML('<h4>Filtros de análise</h4>'),
    regiao, limiar
])
display(painel)
```

Não dependa somente da cor para comunicar estados; prefira rótulos, texto e valores numéricos. Evite interfaces que exigem múltiplas rolagens ou possuem descrições truncadas.

8. Filtragem de dados e renderização reativa

A interação melhora quando um controle corresponde a uma pergunta. Exemplo: selecionar uma categoria de vendas e calcular sua receita agregada. A função de cálculo deve ser pura, sem alterar o DataFrame de origem.

```
import pandas as pd
import ipywidgets as widgets
from IPython.display import display

dados = pd.DataFrame({'categoria':['A','B','A'],
                      'receita':[120,80,150]})
seletor = widgets.Dropdown(options=['Todas','A','B'])
saida = widgets.Output()

def atualizar(change=None):
    base = dados if seletor.value == 'Todas' else \
           dados[dados['categoria'] == seletor.value]
    with saida:
        saida.clear_output(wait=True)
        print('Receita:', int(base['receita'].sum()))

seletor.observe(atualizar, names='value')
display(widgets.VBox([seletor,saida]))
atualizar()
```

O cálculo é pequeno. Para tabelas extensas, debouncing e caching podem ser necessários, e nem todo evento deve executar uma nova consulta.

9. Uploads, entrada do usuário e segurança

`FileUpload` permite receber arquivos por interação. A estrutura do valor retornado pode depender da API e do frontend; trate o conteúdo como entrada não confiável e valide tamanho, extensão e formato real antes de processar.

Cuidados essenciais: não execute código vindo de uploads; não salve arquivos recebidos com caminhos escolhidos livremente pelo usuário; limite quantidade e tamanho; remova dados pessoais antes de compartilhar notebooks.

Para dados sensíveis, prefira armazenamentos apropriados fora do notebook e exclua outputs que possam expor informações. Um `.ipynb` versionado pode carregar resultados de células, inclusive texto digitado anteriormente.

10. Erros comuns e diagnóstico

Sintoma	Causa provável	Diagnóstico
Controle aparece como texto	Frontend sem integração	Ver extensão, atualização e reinício
Callback executa duas vezes	Observadores duplicados	Rever registros e recriação de células
Tela fica lenta	Cálculo em toda mudança	Botão, cache e redução de eventos
Valor inesperado	Estado desatualizado	Ver `widget.value` e ordem de células
`Output` sem resposta	Exceção no callback	Registrar falha e testar função separada
Widget muda, gráfico não	Atualização não conectada	Verificar callback e renderização

Consulte a configuração do kernel e do frontend antes de reinstalar bibliotecas. Se o Python executa e apenas a visualização falha, a investigação deve começar pela integração de widgets.

11. Testes: lógica separada da interface

Crie funções para cálculo que recebam parâmetros explícitos e retornem resultados. Teste essas funções com `assert`; o callback deve apenas ler controles, chamar a função e atualizar a saída.

```
def receita_categoria(registros, categoria):
    filtrados = (r for r in registros
                 if categoria == 'Todas' or r['tipo'] == categoria)
    return sum(r['valor'] for r in filtrados)

itens = [{'tipo':'A','valor':10},{'tipo':'B','valor':8}]
assert receita_categoria(itens, 'A') == 10
assert receita_categoria(itens, 'Todas') == 18
```

Isso reduz o acoplamento com o JupyterLab e permite reutilizar a análise posteriormente em scripts ou aplicações.

12. Consulta rápida: APIs mais frequentes

Objetivo	Componente ou API
Controle de valor inteiro	`widgets.IntSlider`
Lista suspensa	`widgets.Dropdown`
Ação explícita	`widgets.Button` + `.on_click`
Escutar mudanças	`.observe(callback, names='value')`
Remover observador	`.unobserve(callback, names='value')`
Organizar verticalmente	`widgets.VBox([...])`
Capturar saída	`widgets.Output()`
Sincronizar valores	`widgets.link(...)`
Protótipo declarativo	`ipywidgets.interact`

13. Oficina orientada: painel de exploração

Construa um painel com um seletor de categoria, um limiar numérico e um botão “Aplicar filtros”. O resultado deve mostrar contagem, soma e média, tratando o caso de nenhum registro encontrado. Separe cálculo, validação e apresentação. Acrescente uma célula Markdown explicando a pergunta respondida.

Critérios de aceite: a função de cálculo funciona sem widgets; não existem callbacks duplicados; o botão gera resultado apenas após clique; estados vazios são documentados; o notebook executa do início ao fim após reiniciar o kernel.

14. Aprofundamento: desenho de uma aplicação reativa

Uma interface reativa possui **entradas**, **estado**, **processamento** e **saídas**. O melhor desenho depende de separar essas responsabilidades. Um controle deve informar claramente o que altera; a função de cálculo não deve precisar conhecer o widget; e o feedback deve diferenciar estado vazio, carregamento, resultado e erro.

Separar função pura de interação

```
def classificar(valor, limite):
    return 'alto' if valor >= limite else 'baixo'

assert classificar(5, 6) == 'baixo'
assert classificar(6, 6) == 'alto'
```

O teste não precisa do JupyterLab. O callback pode apenas coletar `valor` e `limite`, chamar `classificar` e atualizar um `HTML` ou `Output`. Isso facilita manutenção, evita repetir lógica e torna os resultados mais auditáveis.

Validação antecipada: se um controle representa porcentagem, limite seus valores e informe a unidade no rótulo. Se um campo aceita texto numérico, trate falhas de conversão em vez de deixar uma exceção interromper a interação silenciosamente.

15. Aprofundamento: desempenho, acessibilidade e portabilidade

Widgets que disparam uma leitura de arquivo ou consulta grande a cada alteração podem tornar o notebook lento. Use filtros aplicados por botão, cálculo incremental ou resultados em cache quando a tarefa justificar. Evite atualizações excessivas de imagens grandes.

As interações precisam considerar pessoas usando teclado, leitor de tela ou dispositivos com telas pequenas. Prefira rótulos curtos e objetivos, valores visíveis e confirmação textual dos resultados. Não use o tom de cor como única indicação de erro.

Portabilidade: um notebook que mostra widgets no JupyterLab pode não preservar a interação quando exportado como PDF ou visualizado estaticamente no GitHub. Explique os parâmetros e apresente ao menos um resultado representativo em texto ou tabela, além dos controles interativos.

Checklist para publicação

1. Reinicie o kernel e execute as células em sequência.
2. Confira se os callbacks não foram registrados duas vezes.
3. Teste valores mínimos, máximos, entradas vazias e categorias inexistentes.
4. Assegure que o painel comunica falhas de forma compreensível.
5. Não salve uploads sensíveis dentro do notebook.
6. Verifique a aparência com o frontend de destino e documente limitações.

16. Exercícios e leituras

1. Compare `interact` e `observe` quanto ao controle de execução.
2. Demonstre quando usar `Button` em vez de executar a cada mudança.
3. Crie um seletor com opção “Todas” e três categorias.
4. Registre e remova um observador, verificando o resultado.
5. Descreva os riscos de processar diretamente um arquivo recebido por upload.
6. Explique por que o cálculo deve ser separado do callback.

Notebook correspondente: `03_Widgets_e_Interatividade.ipynb`.

- ipywidgets: https://ipywidgets.readthedocs.io/
- Traitlets: https://traitlets.readthedocs.io/
- IPython display: https://ipython.readthedocs.io/
- JupyterLab: https://jupyterlab.readthedocs.io/

Este material é autoral e foi escrito para acompanhar o notebook sem substituir suas atividades.